Privacy-Preserving Machine Unlearning in Federated LLMs: A Certified Right-to-Be-Forgotten Framework

G.SHASHANK
Roll No. 23K91A6761
Dept. of CSE (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

B.SATHISH
Roll No. 23K91A6719
Dept. of CSE (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

G.RAKESH
Roll No. 23K91A6759
Dept. of CSE (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

B.AKHIL
Roll No. 23K91A6721
Dept. of CSE (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

[GUIDE NAME]
[DESIGNATION], Project Guide
Dept. of CSE (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

***Abstract*—**Federated language models keep raw data on client devices, yet every client's influence stays in the shared parameters, so a right-to-be-forgotten request can be met only by retraining or by an unverifiable heuristic. This paper presents ForgetSafe, a reproducible framework for certified client-level unlearning. Each client's clipped parameter update is tracked; a deletion request subtracts it from the noisy aggregate, and the result is checked against a retained-only recomputation, a membership-inference attack, a utility measure and a hash-chained audit ledger, which together issue a certificate. The method is measured on synthetic data with a bigram proxy, a larger synthetic corpus and LoRA-style adapters of a small neural language model. Removal is exact (distance 3e-14) and matches a fresh-noise retrain in distribution; without noise the attack AUC falls from 0.99 to 0.54 and, for adapters, from 0.78 to 0.53. The default noise ($\varepsilon = 1.61$) leaves the five-client model worse than uniform, whereas $\varepsilon < 1$ and a useful model coexist once the federation reaches about 1,000 clients. Load, security and acceptance tests expose 8 defects in the service.

***Index Terms*—**audit trail, differential privacy, federated learning, machine unlearning, membership inference, right to be forgotten.

I. INTRODUCTION

Machine unlearning for federated large language models (LLMs) is a young field. Following the review method of the base paper [1], which organises a research area into a taxonomy, compares the strengths and limits of each class and states the open gap, this paper surveys the area (Section II) and then builds and measures a certified-unlearning framework (Sections III and IV). The base paper reviews sign-language recognition, so only its method, not its subject, is used.

In federated learning, clients train locally and send only parameter updates to a server [2]; parameter-efficient adapters such as LoRA [3] make this practical for LLMs. Raw text never leaves a client, but the client's influence stays in the shared parameters, and large models can reproduce training text [4]. Data-protection law gives individuals a right to erasure [5], so a client may demand that its contribution be removed. Today this is done by retraining from scratch, which is exact but costs a full training run per request; by retraining only affected shards [6]; or by approximate weight edits that come with no proof [7]. Certified removal exists for convex, centralised models [8, 9], but a federated pipeline that removes a client, proves the removal and leaves an audit record that a regulator can check is still missing.

The study asks four questions. Does removal by subtraction give the same model as retraining? Does the certificate's own check mean anything? What do the differential-privacy noise and its bound do to utility and to leakage? And can the audit ledger be trusted? All answers are measured on synthetic data and can be recomputed on an ordinary PC.

The contributions of this work are:

1) a tracked-update workflow: clipped client updates are stored, and removal is the subtraction of one stored update, with no raw data and no retraining;
2) a verification stack that issues a certificate per request: retained-only recomputation, a loss-based membership attack, utility retention and a SHA-256-chained ledger;
3) an evaluation that tests the certificate itself on three models (a bigram proxy, a larger synthetic corpus and LoRA-style neural adapters) with fresh-noise retraining, noise-free attacks, a noise sweep and an audit-tamper matrix;
4) measurements of the service (load, concurrency, 58 hostile requests, scripted acceptance) and of the federation size at which $\varepsilon < 1$ and a useful model coexist.

Section II surveys related work, Section III describes the method, Section IV reports measurements and Section V concludes.

A. System Hardware

All measurements ran on one workstation: Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, Windows 11 Pro. No GPU was used.

B. Software Requirements

The engine and server use only the Python 3.11.9 standard library (http.server, hashlib, unittest); the neural study uses PyTorch 2.14 (CPU). The interface is React 19 with Vite 8 and Recharts on Node.js 24.19; screens and acceptance runs use Playwright in Chrome.

II. LITERATURE SURVEY

A. Base Paper

Khan et al. [1] review deep-learning approaches to continuous sign language recognition (IEEE Access, 2025). They organise the studies into a taxonomy covering sign language, data collection, input method, gesture signals, identification methods and datasets, split the models into spatial, temporal and alignment approaches with their benefits and drawbacks, and close with challenges such as real-time processing, dataset diversity and signer variability. Its subject is unrelated to unlearning, so it offers no result that ForgetSafe could be compared with; what is taken from it is the taxonomy, compare, find-the-gap method used in this section.

B. Related Work

Exact and sharded removal. Cao and Yang [10] make a learner forget by updating the sums it is built from, and SISA training [6] confines retraining to the shard that held the data. *Certified removal.* Guo et al. [8] remove a point from a convex model with a Newton step plus noise and an (ε, δ) guarantee; Sekhari et al. [9] bound how many deletions a learner can absorb. *Federated unlearning.* FedEraser [7] keeps the clients' historical updates on the server and calibrates them to rebuild a model without one client, the idea ForgetSafe reduces to its simplest form. A recent survey [11] compares FedEraser, FedCIO, FedRecovery and F2UL, finds a trade-off between removal precision and cost, and calls for standard evaluation metrics. *Recent systems.* FedMemPrune [12] argues that unlearning should remove only the information unique to the forgotten data and resets the parameters that memorise it. Oblivionis [13] unifies federated learning and unlearning for LLMs and compares six federated and five unlearning algorithms. FedORA [14] casts removal in vertical federated learning as a primal-dual problem with a bound to retraining. *Privacy tools.* The Gaussian mechanism [15] and its tighter calibration [16] give the (ε, δ) bounds, and loss-based membership inference [17, 18] is the standard empirical check. Three further items of the project's literature list could not be retrieved and are not used.

Strengths and limits. Retraining and sharding give exact removal, but cost grows with model size and request rate. Certified removal gives a formal guarantee, but only for convex models. Historical-update methods are cheap but approximate. The memorisation-aware and primal-dual methods target neural networks and report agreement with retraining on benchmarks, not a per-request proof, and the LLM framework [13] concentrates on measuring forgetting and utility rather than on evidence a regulator can check. A differential-privacy statement describes a release mechanism and says nothing about forgetting unless the noise and the accounting are handled again after the removal.

TABLE I
RELATED WORK COMPARED

Work	Approach	Data	Reported result	Limitation
Khan [1]	Taxonomy review of sign-language models	Sign-language datasets	None on unlearning	Unrelated subject; method only
FedMemPrune [12]	Prune parameters that memorise	CIFAR-10/100, EMNIST	Close to retraining baselines	Image models, not LLMs
Oblivionis [13]	FL + unlearning framework, 6 + 5 algorithms	Llama-2/3 models; TOFU, MUSE	Beats local training; balance of forgetting and utility	No audit or certificate stage described
FedORA [14]	Primal-dual removal, bound to retrain	Five tabular and image sets (vertical FL)	Comparable to retraining at lower cost	Vertical FL; no audit record
Zhang survey [11]	Review of FedEraser, FedCIO, FedRecovery, F2UL	Review (no new data)	Precision-cost trade-off	Review only; no system
Guo [8]	Newton step + noise	Convex models	(ε, δ) removal guarantee	Not federated, not LLM
ForgetSafe (this work)	Tracked-update subtraction + certificate + ledger	Synthetic: 5 bigram clients; 20-domain corpus; 8 neural-adapter clients	Exact removal; certificate tested (Sec. IV)	Linear one-round aggregate; small models

Gap. The works above remove influence heuristically, with an empirical or a convex-model guarantee; none ties a federated removal to a certificate, an independent check and a tamper-evident record, and none asks whether such a certificate is itself sound. ForgetSafe provides the first and tests the second. The project's survey lists the same practice and its limits: full retraining is too expensive for large federated LLMs, heuristics give no formal guarantee, sharding adds complexity, differentially private training costs utility, and unlearning is mostly neither verifiable nor auditable. Those limits frame the four questions in Section I.

III. METHODOLOGY

A. Pipeline

Fig. 1 shows the system and Fig. 2 the main flow. A Python standard-library server (port 8109) holds one engine; a React interface (port 5109) calls nine JSON endpoints. Training builds the noisy global model; an unlearning request removes one client and runs the checks; every training and unlearning event is appended to the ledger.

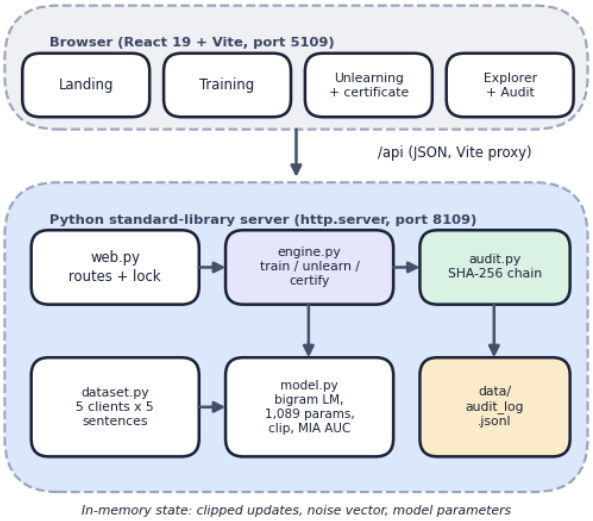


Fig. 1. ForgetSafe architecture: React interface, standard-library server, engine, model and hash-chained ledger.

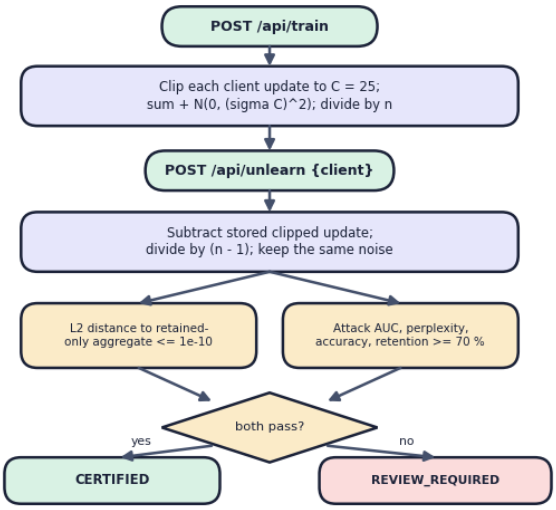


Fig. 2. Main flow: train, unlearn, two checks, certificate status.

B. Data and Models

Bigram proxy. Five simulated clients (Finance, Healthcare, Education, Retail, Travel) hold five hand-written sentences each; five public sentences measure utility and five unseen sentences serve as non-members. Text is lower-cased to a 33-symbol alphabet. A client's update u is the vector of its 33 × 33 = 1,089 bigram counts, and the next-character distribution is $P(c'|c) \propto \max(0, \theta[c, c']) + 0.15$.

Synthetic corpus. A seeded template grammar with 20 domains gives each client 200 training and 200 held-out sentences and a 300-sentence public set; the engine classes run unchanged. A raw update of such a client has norm about 1,239, so C = 25 would shrink it 50-fold; C is set to the median norm. The domain-level attack scores a client's training sentences against other clients' held-out sentences; the sample-level attack uses its own held-out sentences.

Neural adapters. A character-level GRU (137,151 parameters) is pre-trained on a synthetic public pool and frozen. Each of eight synthetic clients trains a rank-64 LoRA-style adapter [3] (14,272 parameters) on 150 sentences of its own grammar and vocabulary. The adapter vectors are clipped at the median raw norm (C ≈ 13), summed, noised and divided by n exactly as the bigram counts are, and removal subtracts the stored adapter.

C. Formulation

With clip norm C, noise multiplier σ and n clients, training releases

$$\hat{u}_i = u_i \cdot \min(1, C / \|u_i\|) \quad (1)$$

$$\theta = (\Sigma_i \hat{u}_i + z) / n, \quad z \sim N(0, \sigma^2 C^2 I) \quad (2)$$

$$\varepsilon = \sqrt{(2 \ln(1.25/\delta))} / \sigma, \quad \delta = 1e\text{-}5 \quad (3)$$

giving ε = 1.615 at C = 25, σ = 3. A request for client j stores nothing new and computes

$$\theta_{-j} = (\Sigma_i \hat{u}_i + z - \hat{u}_j) / (n - 1) \quad (4)$$

$$\Delta_j = \| \theta_{-j} - (z + \Sigma_{i \neq j} \hat{u}_i) / (n - 1) \|_2 \leq 1e\text{-}10 \quad (5)$$

Because the aggregate is linear, Δ is zero up to rounding: the certificate's reference check is an identity, not an independent retrain, since it reuses the stored noise z. The status is CERTIFIED only if Δ passes and utility retention stays at or above 70 %, where utility = 0.65·accuracy + 0.35/(1 + ln(1 + perplexity)) and retention is the ratio after to before.

D. Why This Algorithm

Subtracting a stored update needs no raw data, costs O(d) in the parameter count d, and is exact for a linear aggregate; the price is that the server must store every client's clipped update, itself a sensitive artefact. For multi-round nonlinear fine-tuning an update influences later updates, so exact subtraction no longer holds and methods such as [7, 12] become necessary. Adding or removing one client changes Σû by at most C in L2, so noise of standard deviation σC per coordinate is the Gaussian mechanism [15] with sensitivity C, and (3) is its classical calibration, formally valid only for ε < 1. The unlearned model reuses z: it is a second function of the same noisy release, not a fresh one, so ε in (3) is reported unchanged after a removal and is not, by itself, a statement about forgetting.

E. Verification, Algorithms and Audit

The attack scores a text by its negative loss and reports an AUC [17, 18]; 0.5 means no leakage. Each ledger record stores the SHA-256 of the previous hash and its canonical JSON, so verification re-walks the chain. A certificate has 25 fields, among them status, ε, δ, C, σ, the reference distance, the attack AUC, perplexity and accuracy before and after, retention, model hashes and the hash of its own ledger record.

$$h_k = \text{SHA-256}(h_{k-1} \mid \text{canonical}(r_k)), \quad h_0 = 64 \text{ zeros} \quad (6)$$

Training: count and clip each client's update, store it with its raw norm, draw z once from the session seed, keep S = Σû + z, release θ = S/n and append a FEDERATED_TRAINING record. *Unlearning client j:* reject if untrained, unknown, already removed or last; measure metrics and attack; set S ← S − û_j and θ ← S/(n − 1); measure again; compute Δ; set the status; append a CLIENT_UNLEARNING record holding every certificate field; return the certificate. The server is trusted to hold the updates; malicious clients, secure

aggregation and an adversary with write access to the ledger file are out of scope.

F. Evaluation Protocol

Every experiment runs the real engine on a throw-away ledger with seeded randomness. *Bigram proxy:* 50 seeds × 5 clients (250 requests); 12 noise values (20 seeds); 200 fresh-retrain pairs at σ = 3 and 0.5; a noise-free run; scaling from 5 to 500 synthetic clients; seven ledger edits. *Synthetic corpus:* n = 5, 10, 20 clients, σ from 1e-9 to 10 (10 seeds × 5 targets); n up to 4,000 for the ε < 1 question. *Neural adapters:* 8 clients, 9 noise values, 5 seeds. *Service:* 1 to 64 concurrent clients, three race tests, a 2,000-request soak, 58 hostile or malformed requests and 20 scripted synthetic-user browser journeys.

IV. RESULTS

Conditions. Default configuration is seed 2026, C = 25, σ = 3, δ = 1e-5 on the bigram proxy; seed studies use 50 seeds × 5 clients (250 requests). Perplexity is on public sentences (5 for the proxy); AUC rows for the corpus and adapters are domain or client level. A model is 'destroyed' when its perplexity exceeds 1e8.

TABLE II
MEASURED UNLEARNING RESULTS (MEAN OVER REQUESTS; ADAPTER ROWS USE RETAINED-CLIENT PERPLEXITY AND CLIENT-LEVEL AUC)

Setting	ε	Perplexity	Accuracy	AUC before → after	Retention %
Default, seed 2026 (n = 5)	1.61	130.2	5.1 %	0.22 → 0.21	96.7
Default, 50 seeds (250 requests)	1.61	142.2±28	3.7 %	0.52 → 0.50	97.1
Bigram σ = 0.1 (20 seeds)	48.4	16.3	23.5 %	0.86 → 0.55	96.5
Bigram σ = 4.85 (20 seeds)	1.00	197.9	3.4 %	0.54 → 0.53	97.7
Bigram, noise-free σ = 1e-6	≈ 5e6	12.4	n/a	0.99 → 0.54	99.7
Corpus n = 10, σ = 0.1	48.4	12.7	33.2 %	0.47 → 0.32	99.0
Corpus n = 1,000, σ = 4.85	0.999	10.6	33.6 %	0.40 → 0.40	99.998
Adapters σ = 0 (n = 8)	∞	9.2	47.9 %	0.78 → 0.53	99.9
Adapters σ = 0.05 (n = 8)	96.9	12.6	46.0 %	0.76 → 0.53	92.1
Adapters σ = 0.1 (n = 8)	48.4	47.3	39.5 %	0.74 → 0.56	59.2
Adapters σ ≥ 0.3	≤ 16	destroyed	≤ 8.1 %	0.46 → 0.41	n/a

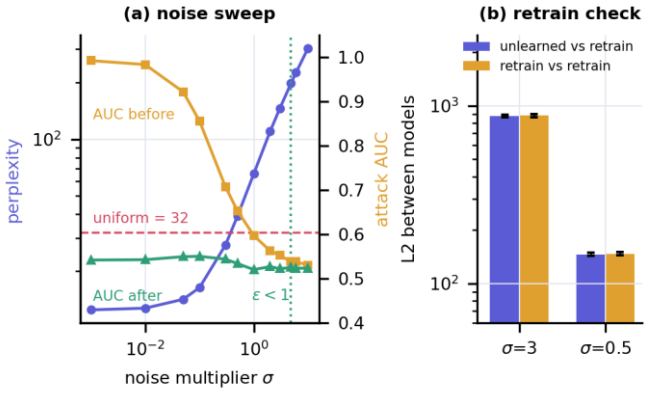


Fig. 3. (a) Perplexity and attack AUC against the noise multiplier (bigram, 20 seeds). (b) L2 distance of the unlearned model to a fresh-noise retrain and between two retrains (200 pairs).

Removal and utility. On the proxy, removal is exact to 3.2e-14 and all 250 requests are certified. Retention averages 97.1 % (minimum 79.9 %); removing four of five clients raises perplexity from 130 to 303 because the noise stays while the signal is divided by fewer clients, every step is certified, and the fifth removal is refused. With five members and five non-members one pair moves the AUC by 0.04, so the per-client AUC values at the default seed (0.04 to 0.48 before removal) carry no information.

The default model is noise-dominated. The noise has standard deviation 75 per coordinate against a clipped update of norm 25 spread over 1,089 coordinates. Perplexity is 142, worse than the uniform model's 32, accuracy is 3.7 %, and the attack is at chance before removal (AUC 0.52 ± 0.22), so there is nothing for removal to remove. On five clients, useful models need σ ≤ 0.1 (ε ≥ 48), whereas the classical bound is valid only for ε < 1 (σ > 4.845), where perplexity is 198 or more. Because retention is measured against a model that is already near-useless, the 70 % floor cannot fail at σ = 3; it failed once in the sweep, at σ = 0.3.

Is the removal real? Without noise the attack separates members almost perfectly (AUC 0.99) and removal brings it to 0.54 with 99.7 % retention. Against a true retrain with fresh noise the unlearned model lies 875 away at σ = 3 (146 at σ = 0.5), the same as two independent retrains (881 and 147): the distance is pure noise, so the unlearned model is distributed like a retrained one, which the 1e-14 certificate value alone does not show.

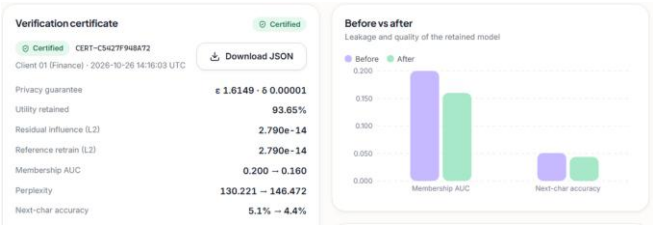


Fig. 4. Output screen: certificate for Client 01 (Finance), ε 1.6149, AUC 0.200 → 0.160, retention 93.65 %.

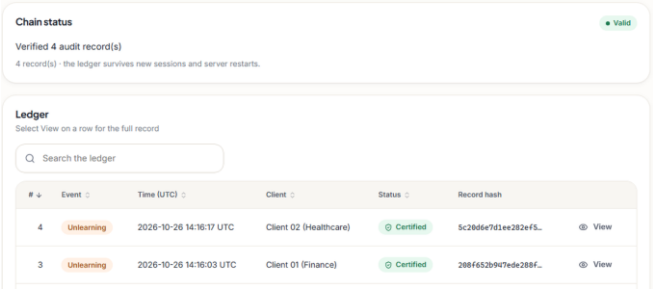


Fig. 5. Output screen: compliance ledger, chained records and a valid hash chain.

Larger data and neural adapters. With 200 sentences per client the bigram model still does not memorise sentences (sample-level AUC 0.50 at every σ) but leaks the domain: the attack falls from 0.474 to 0.220 on removal without noise and from 0.472 to 0.324 at σ = 0.1, and unlearned and fresh-retrain models are 645 and 650 apart. Since the noise is added once to the sum and divided by n, a large federation dilutes it: at σ = 4.85 (ε = 0.999) perplexity is 1.17 times the noise-free value at n = 1,000 clients and 1.03 at 4,000 (Fig. 6a), against 102 at n = 5, and five sequential removals at n = 1,000 are certified. For the neural adapters a single adapter is plainly attackable (client-level AUC 1.00, sample-level 0.74); the released average hides samples but not the domain (0.78), and removal returns the attack to 0.53, equal to a fresh

retrain (0.53), with unlearned-to-retrain distance equal to retrain-to-retrain at every σ (31.8 and 31.8 at σ = 0.1). Retention is 99.9 % at σ = 0, 92 % at 0.05 and 59 % at 0.1 (below the floor, so REVIEW_REQUIRED), and the models are destroyed from σ = 0.3 (Fig. 6b). Subtracting an adapter takes 24 µs, the fully checked request 91 ms, and retraining the seven retained adapters 5.9 s.

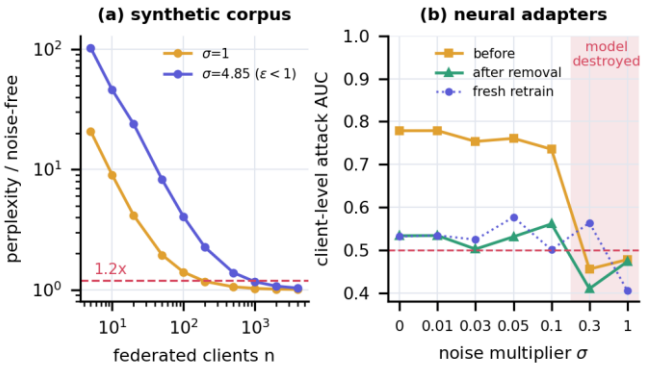


Fig. 6. (a) Perplexity relative to the noise-free model against federation size (synthetic corpus). (b) Client-level attack AUC against σ for neural adapters.

Audit, service and tests. The ledger detects an edited field, a flipped status, a deleted record and swapped records, but not tail truncation, a re-hashed forgery or an appended forged record, and an emptied ledger verifies as valid; an append re-reads the file (0.44 ms at one record, 19.6 ms at 2,000). On the proxy, unlearning (45 ms) is slower than re-running the training (13 ms) with 5 clients and 6.8 times faster at 500. The five unit tests and 14 smoke checks pass. From 1 to 64 concurrent clients the server returns no 5xx error at up to 8 clients; at 16 to 64 a handful of requests fail (at most 19 of 500), and /api/state saturates near 86 requests/s because it re-reads the ledger. 32 simultaneous removals of one client yield exactly one success, 16 simultaneous trainings write 16 valid records, and a 2,000-request soak has no error. Of 58 hostile or malformed requests 35 are handled, 15 behave noteworthily and 8 are defects: no read timeout and a negative Content-Length block a thread, a cross-site POST and a forged Host header are accepted, burst connections are refused, a corrupt ledger line breaks the interface, and no endpoint is authenticated. All 20 scripted synthetic-user journeys in Chrome (11 desktop, 9 phone, 14 dark mode; 49 certificates) pass with no console error; this is a scripted test, not a human study.

Comparison. The base paper [1] reports no comparable metric. FedMemPrune [12], Oblivionis [13] and FedORA [14] use image, benchmark-LLM and vertical-federated data with neural models, so none of the numbers above is comparable with theirs; the comparison is qualitative: ForgetSafe is exact because its aggregate is linear and single-round, whereas those works handle nonlinear networks, and it adds a per-request certificate and a tamper-evident ledger.

V. CONCLUSION

ForgetSafe meets its six objectives at the scale it was built for. Federated aggregation runs across five bigram clients and eight clients fine-tuning LoRA-style adapters of a 137k-parameter GRU; removal needs no retraining (24 µs for an adapter against 5.9 s); the (ε, δ) bound is reported, but at the default σ it lies outside the classical range, the noise is reused, no accountant exists, and ε < 1 with a useful model needs about 1,000 clients; the membership attack and the fresh-noise retrain agree on all three models; retention is 97.1 % at the default and 99.9 % for adapters without noise; and every event is chained, with the blind spots and service defects measured above. The key finding is that the certificate proves the exactness of the arithmetic, not forgetting: the noise-free, fresh-noise and domain-level experiments supply that evidence. The study covers a 137k-parameter GRU on synthetic data in one aggregation round on one CPU workstation, and makes no claim about larger pretrained LLMs or multi-round training.

Future work: draw fresh noise or keep a privacy accountant [16] after each removal; sign and externally anchor the ledger and add authentication, request timeouts and origin checks; extend the adapters to a pretrained language model over several rounds [3]; and test stronger membership attacks. In short, removal by subtraction equals the retained-only aggregate exactly and a fresh retrain in distribution, the certificate's own check is an identity, the default noise removes the leakage and the utility together, and the ledger resists casual edits but not an adversary who can rewrite the file.

REFERENCES

[1] A. Khan, S. Jin, G.-H. Lee, G. E. Arzu, L. M. Dang, T. N. Nguyen, W. Choi, and H. Moon, "Deep learning approaches for continuous sign language recognition: A comprehensive review," IEEE Access, vol. 13, pp. 55524-55544, 2025, doi: 10.1109/ACCESS.2025.3554046.

[2] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Aguera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS), PMLR vol. 54, 2017, pp. 1273-1282.

[3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in Proc. Int. Conf. Learning Representations (ICLR), 2022.

[4] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, A. Oprea, and C. Raffel, "Extracting training data from large language models," in Proc. 30th USENIX Security Symp., 2021, pp. 2633-2650.

[5] European Parliament and Council, "Regulation (EU) 2016/679 (General Data Protection Regulation)," Official Journal of the European Union, L 119, May 2016, Art. 17 (right to erasure).

[6] L. Bourtoule, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, "Machine unlearning," in Proc. IEEE Symp. Security and Privacy (SP), 2021, pp. 141-159.

[7] G. Liu, X. Ma, Y. Yang, C. Wang, and J. Liu, "FedEraser: Enabling efficient client-level data removal from federated learning models," in Proc. IEEE/ACM 29th Int. Symp. Quality of Service (IWQoS), 2021, pp. 1-10.

[8] C. Guo, T. Goldstein, A. Hannun, and L. van der Maaten, "Certified data removal from machine learning models," in Proc. 37th Int. Conf. Machine Learning (ICML), PMLR vol. 119, 2020, pp. 3832-3842.

[9] A. Sekhari, J. Acharya, G. Kamath, and A. T. Suresh, "Remember what you want to forget: Algorithms for machine unlearning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 34, 2021, pp. 18075-18086.

[10] Y. Cao and J. Yang, "Towards making systems forget with machine unlearning," in Proc. IEEE Symp. Security and Privacy (SP), 2015, pp. 463-480.

[11] W. Zhang, "A comprehensive investigation of federated unlearning: Challenges, methods and future prospects in privacy-sensitive applications," in Proc. 2nd Int. Conf. Data Analysis and Machine Learning (DAML), SciTePress, 2024, pp. 569-573, doi: 10.5220/0013528500004619.

[12] J. Wei, Y. Zhang, H. Zhang, L. Y. Zhang, C. Chen, K.-L. Ong, J. Zhang, and Y. Xiang, "Rethinking federated unlearning via the lens of memorization," in Proc. 32nd ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD '26), Jeju Island, South Korea, Aug. 2026, doi: 10.1145/3770855.3817785; arXiv:2605.24545.

[13] F. Zhang, X. Yan, T. Wu, W. Li, T. Chen, Y. Cao, R. Yan, L. Huang, W. Y. B. Lim, and Q. Yang, "Oblivionis: A lightweight learning and unlearning framework for federated large language models," arXiv:2508.08875, Aug. 2025.

[14] Y. Jiang, X. Tong, Z. Liu, X. Zhang, K.-Y. Lam, and C. W. Tan, "Certifying the right to be forgotten: Primal-dual optimization for sample and label unlearning in vertical federated learning," arXiv:2512.23171, 2025.

[15] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," Foundations and Trends in Theoretical Computer Science, vol. 9, no. 3-4, pp. 211-407, 2014.

[16] B. Balle and Y.-X. Wang, "Improving the Gaussian mechanism for differential privacy: Analytical calibration and optimal denoising," in Proc. 35th Int. Conf. Machine Learning (ICML), PMLR vol. 80, 2018, pp. 394-403.

[17] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in Proc. IEEE Symp. Security and Privacy (SP), 2017, pp. 3-18.

[18] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in Proc. IEEE 31st Computer Security Foundations Symp. (CSF), 2018, pp. 268-282.